

Development of AI Response Validation System with Hallucination Detection Assistance

EvalAI - AI Response Quality Evaluator Engine

Technical Documentation — A multi-agent LLM evaluation system with RAG grounding, batch processing, and analytics

Prepared for: Infosys Springboard Virtual Internship — 7.0

Author: Ghayathri Devi G M

1. Project Overview

The EvalAI - AI Response Quality Evaluator is a multi-agent system that evaluates the quality of AI-generated responses to a given question. Rather than trusting a response at face value, the platform runs it through four independent LLM-as-judge agents — Relevance, Accuracy, Completeness, and Hallucination Detection — grounded against retrieved reference material, and aggregates their findings into a single weighted verdict.

The system supports both single-response evaluation and bulk CSV-based batch evaluation, persists every evaluation to a hosted database, and provides an analytics dashboard for tracking quality trends across many evaluations over time. Results can be exported as structured PDF reports.

2. Problem Statement

As AI systems are increasingly used to generate answers to user questions, there is no reliable, automated way to check whether a given response is actually good — whether it answers what was asked, whether it is factually correct, whether it covers everything the question required, and whether it invents information not supported by any real source.

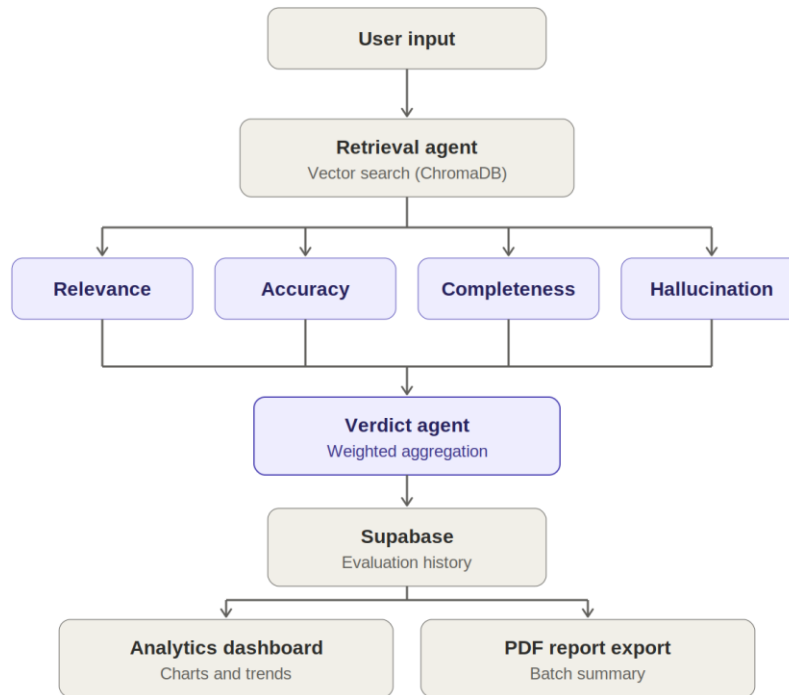
Manually reviewing AI responses for these properties does not scale. This project addresses that gap by building an automated evaluation pipeline that judges AI responses across four distinct quality dimensions, cites its reasoning for each judgment, and flags fabricated claims explicitly rather than producing an opaque single score.

3. Objectives

- Build a Retrieval-Augmented Generation (RAG) pipeline to ground evaluations in real reference material, not just the AI judge's own opinion.
- Build four independent judge agents (Relevance, Accuracy, Completeness, Hallucination Detection), each producing a score and human-readable reasoning.
- Aggregate the four agents' outputs into a single weighted verdict, reflecting that factual correctness and grounding matter more than surface-level relevance or completeness.
- Support both single-submission and bulk batch (CSV) evaluation modes.
- Persist all evaluations for historical tracking and analytics.
- Provide a scoring dashboard visualizing pass/fail rates, per-dimension trends, and hallucination frequency.
- Allow exporting batch evaluation results as a structured PDF report with improvement recommendations.
- Validate the system end-to-end, including agent scoring consistency and error handling.

4. System Architecture

The system follows a layered architecture: an input layer (single form or CSV upload), a retrieval layer (ChromaDB vector search), four parallel evaluation agents, an aggregation layer (the Verdict Agent), a persistence layer (Supabase), and presentation layers (dashboard UI, analytics charts, PDF export).



Two entry points feed this same pipeline: the single-evaluation form (/evaluate) and the batch CSV upload (/batch-evaluate), which runs each row through the identical agent pipeline, with the four independent agent calls per row executed concurrently via a thread pool for performance.

5. Folder Structure

```

ai-eval/
|-- main.py           FastAPI app, all API endpoints
|-- database.py       Supabase read/write layer
|-- retrieval_agent.py RAG retrieval (ChromaDB query)
|-- knowledge_builder.py One-time script: builds the ChromaDB index
|-- relevance_agent.py Relevance Judge Agent
|-- accuracy_agent.py  Accuracy Judge Agent
|-- completeness_agent.py Completeness Judge Agent
|-- hallucination_agent.py Hallucination Detection Agent
|-- verdict_agent.py   Verdict Agent (weighted aggregation)
|-- batch_evaluator.py Batch Evaluation Module (CSV pipeline)
|-- analytics.py       Aggregate statistics for the dashboard
|-- report_generator.py PDF report generation (reportlab)
|-- validate_agents.py Agent validation against SQuAD benchmarks
|-- e2e_test.py        End-to-end test suite
|-- index.html         Dashboard UI (single page, tabbed)
  
```

```
|-- script.js          Frontend logic
|-- style.css          Dashboard styling
|-- chroma_db/         Local vector store (generated by knowledge_builder.py)
|-- .env               API keys (GROQ_API_KEY, SUPABASE_URL, SUPABASE_KEY)
|-- docs/              Agile artifacts (Sprint Backlog, Defect Tracker, Unit Test Plan)
```

6. Installation Guide

6.1 Prerequisites

- Python 3.10 or later
- A Groq API key (console.groq.com)
- A Supabase project with URL and anon key (supabase.com)

6.2 Setup steps

```
git clone <repository-url>
cd ai-eval
python -m venv venv
venv\Scripts\activate      # Windows
source venv/bin/activate   # macOS/Linux

pip install fastapi uvicorn python-multipart pypdf
pip install sentence-transformers==2.7.0 chromadb datasets
pip install langchain-groq python-dotenv json-repair
pip install supabase reportlab matplotlib requests
```

6.3 Environment variables (.env)

```
GROQ_API_KEY=your_groq_key
SUPABASE_URL=https://xxxxx.supabase.co
SUPABASE_KEY=your_supabase_anon_key
```

6.4 Build the knowledge base (one-time)

```
python knowledge_builder.py
```

This downloads SQuAD and a slice of Wikipedia, chunks the text into 100-word segments, embeds each chunk with Sentence-Transformers (all-MiniLM-L6-v2), and stores the result in a local ChromaDB collection named `knowledge_base`.

6.5 Create the Supabase table

Run the following in the Supabase SQL Editor (without Row Level Security, appropriate for a single-user prototype):

```
create table evaluations (  
  id bigint generated always as identity primary key,  
  question text, answer text, reference_answer text,  
  used_source_pdf boolean, retrieved_context jsonb,  
  evaluation_mode text default 'single',  
  accuracy real, accuracy_evidence text, accuracy_supporting_excerpt text,  
  relevance real, relevance_reasoning text,  
  completeness real, completeness_reasoning text, completeness_missing_aspects jsonb,  
  hallucination_risk text, hallucination_flagged_statements jsonb, hallucination_reasoning  
text,  
  overall_score real, verdict_label text, verdict_summary text,  
  created_at timestamptz default now()  
);
```

6.6 Run the server

```
python -m uvicorn main:app --reload
```

Then open index.html directly in a browser. The dashboard communicates with the API at <http://127.0.0.1:8000>.

7. API Documentation

GET /

Health check. Returns a JSON confirmation message that the API is running.

POST /evaluate

Evaluates a single question/answer pair.

Request (multipart/form-data):

- question (string, required)
- answer (string, required)
- reference_answer (string, optional)
- source_pdf (file, optional) — extracted text is appended to retrieved context

Response: JSON containing status, question, retrieved_context, used_source_pdf, and an evaluation object with all four agents' scores/reasoning plus the aggregated verdict.

GET /history

Returns the most recent evaluations (default limit 20), newest first.

POST /batch-evaluate

Accepts a CSV file (multipart/form-data, field name csv_file) with question, answer, and optional reference_answer columns. Evaluates up to 20 rows through the full pipeline, persists each row (tagged evaluation_mode='batch'), and returns per-row results plus an aggregate summary.

GET /analytics

Returns aggregate statistics across stored evaluations: pass/needs-improvement/fail counts, average per-dimension scores, hallucination frequency, verdict distribution, and a day-by-day quality trend. Optional query parameters: start_date, end_date (YYYY-MM-DD), and mode (single or batch).

POST /export-report

Accepts a JSON body of { results, aggregate } (the same shape returned by /batch-evaluate) and returns a downloadable PDF report. Makes no new LLM calls — it only formats already-computed data.

8. Database Schema

A single table, evaluations, hosted on Supabase (PostgreSQL), stores every evaluation from both single and batch modes.

Column	Type	Description
id	bigint	Auto-incrementing primary key
question / answer	text	The submitted question and candidate answer
reference_answer	text	Optional ground-truth answer
used_source_pdf	boolean	Whether an uploaded PDF contributed context
retrieved_context	jsonb	List of retrieved/PDF text chunks
evaluation_mode	text	'single' or 'batch'

accuracy / relevance / completeness	real	0.0–1.0 scores from each agent
accuracy_evidence, accuracy_supporting_excerpt	text	Accuracy agent's reasoning and quoted excerpt
relevance_reasoning	text	Relevance agent's reasoning
completeness_reasoning, completeness_missing_aspects	text / jsonb	Completeness agent's reasoning and unmet aspects
hallucination_risk	text	'Low' / 'Medium' / 'High'
hallucination_flagged_statements, hallucination_reasoning	jsonb / text	Specific flagged claims and reasoning
overall_score, verdict_label, verdict_summary	real / text / text	Weighted verdict output
created_at	timestampz	Insertion timestamp, used for trend charts

9. RAG Workflow

Retrieval-Augmented Generation grounds the Accuracy and Hallucination agents in real source material instead of judging responses in isolation.

9.1 Index construction (`knowledge_builder.py`, run once)

- Loads the SQuAD training set and a slice of English Wikipedia (first 1000 articles).
- Splits every document into fixed 100-word chunks.
- Embeds each chunk using the Sentence-Transformers all-MiniLM-L6-v2 model.
- Stores embeddings and text in a persistent ChromaDB collection (`knowledge_base`).

9.2 Retrieval (`retrieval_agent.py`, runs per request)

- The incoming question is embedded using the same model.
- ChromaDB returns the top-k (default 2) chunks with the closest embedding distance.
- If a PDF was uploaded, its extracted text is appended to the retrieved context, becoming an additional grounding source.

9.3 Usage downstream

Accuracy Agent: if no reference answer is given, retrieved context becomes the fallback ground truth. Hallucination Agent: always checks claims against retrieved context AND the reference answer (if provided) — combining both sources prevents false hallucination flags when retrieval alone misses relevant chunks.

Note: this is retrieval-augmented evaluation, not classic RAG — retrieval feeds into judging an existing answer rather than generating a new one.

10. Agent Descriptions

10.1 Relevance Judge Agent

Scores whether the answer addresses the question asked, independent of factual correctness. Single Groq call (llama-3.1-8b-instant); falls back to local embedding similarity if the LLM call fails twice.

10.2 Accuracy Judge Agent

Checks factual correctness claim-by-claim against the reference answer or retrieved context, classifying each claim as SUPPORTED, UNSTATED (reasonable but unconfirmed), or CONTRADICTED. Only CONTRADICTED claims drive the score toward zero — unstated-but-reasonable claims receive partial credit. Returns a quoted supporting excerpt from the ground truth.

10.3 Completeness Judge Agent

Breaks the question into its distinct required aspects (a question can implicitly ask for multiple things) and checks whether the answer addresses each one, returning a list of specifically missing aspects.

10.4 Hallucination Detection Agent

Identifies specific claims in the answer not supported by retrieved context or the reference answer, flagging the exact unsupported text. Includes a code-level grounding safety net: every LLM-flagged claim is re-verified via string matching (exact and fuzzy) against the source text, automatically correcting false positives where the judge model contradicts itself.

10.5 Verdict Agent

Aggregates the four agents' scores using a weighted formula — Accuracy 35%, Hallucination 30%, Relevance 20%, Completeness 15% — reflecting that factual correctness and grounding matter more than surface relevance or thoroughness. Makes no LLM call of its own; produces a verdict label (Excellent ≥ 0.85 , Good ≥ 0.65 , Needs Improvement ≥ 0.4 , Poor below) and a synthesized summary sentence.

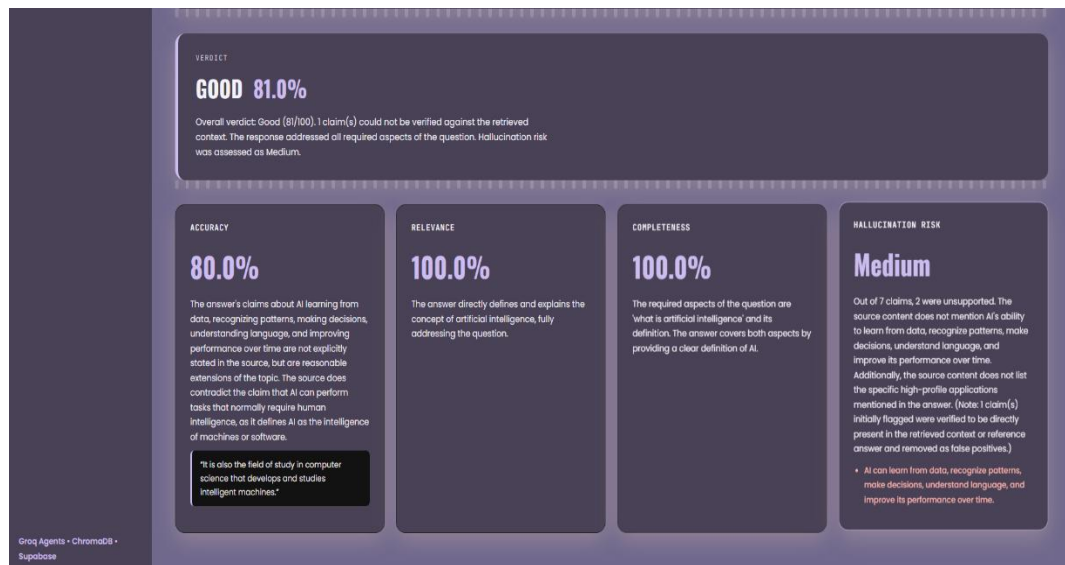
11. Dashboard Explanation

A single-page application with five tabs:

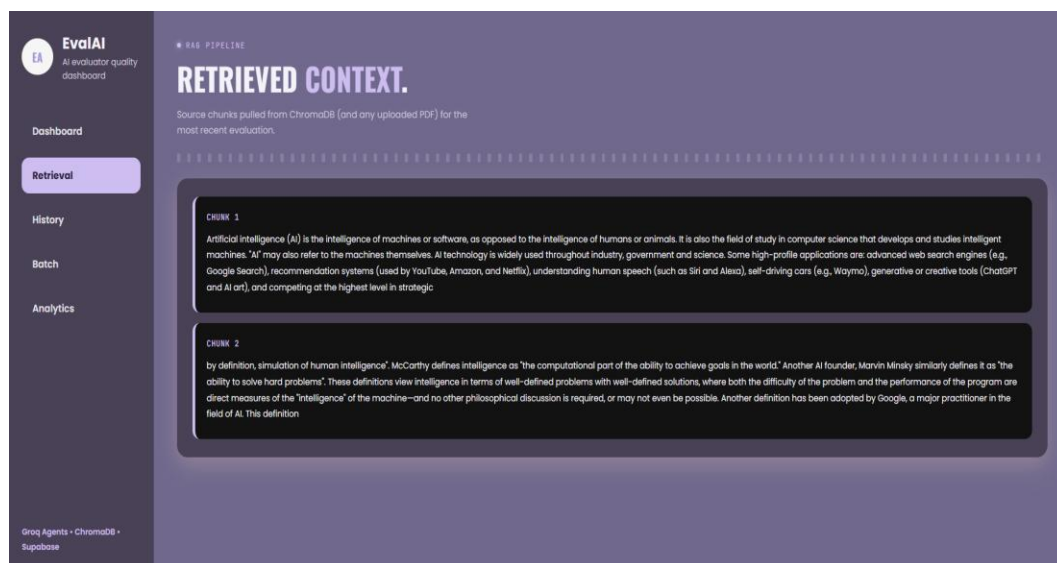
- Dashboard — submission form (question, answer, optional reference/PDF) and the resulting Verdict card plus four per-dimension score cards with reasoning and flagged items.

The screenshot shows the 'EvalAI AI evaluator quality dashboard' with a sidebar containing tabs: Dashboard, Retrieval, History, Batch, and Analytics. The main content area is titled 'AI EVALUATION ENGINE' and 'SUBMIT A RESPONSE. GET THE VERDICT.' It includes a brief instruction: 'Question in, answer in, reference optional. Four agents score relevance, accuracy, completeness, and hallucination risk — a fifth agent weighs them into one verdict.' The form has three sections: 'EVALUATION INPUT' with a 'Question' field containing 'what is artificial intelligence?', a 'Candidate Answer' field containing a paragraph about AI, and a 'Reference Answer (OPTIONAL)' field with the placeholder 'Enter the reference answer, or leave blank...'. A progress bar at the top of the form shows 100% completion.

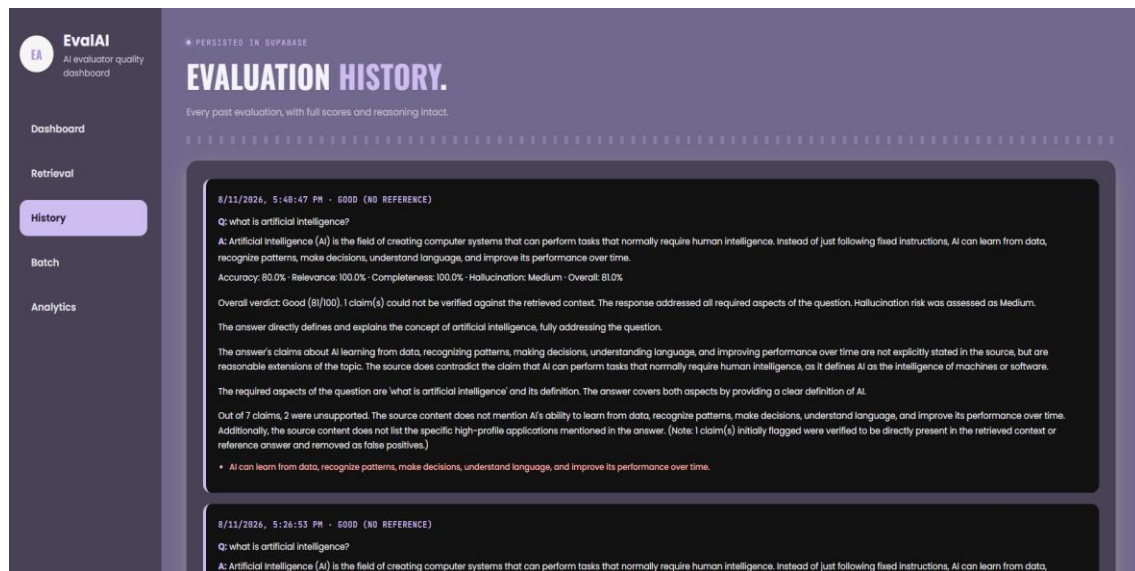
This screenshot shows the bottom portion of the dashboard. It includes the 'Reference Answer (OPTIONAL)' field with the same placeholder. Below it is the 'Source Document (OPTIONAL PDF)' section with a 'Choose File' button and the text 'No file chosen'. An 'Evaluate Response' button is positioned below the source document field. The 'VERDICT' section displays 'GOOD 81.0%' and a summary: 'Overall verdict: Good (81/100). 1 claim(s) could not be verified against the retrieved context. The response addressed all required aspects of the question. Hallucination risk was assessed as Medium.' At the bottom, four score cards are shown: 'ACCURACY 80.0%' (reasoning: 'The answer's claims about AI learning from data, recognizing patterns, making decisions'), 'RELEVANCE 100.0%' (reasoning: 'The answer directly defines and explains the concept of artificial intelligence, fully'), 'COMPLETENESS 100.0%' (reasoning: 'The required aspects of the question are what is artificial intelligence? and its'), and 'HALLUCINATION RISK Medium' (reasoning: 'Out of 7 claims, 2 were unsupported. The source content does not mention AI's ability').



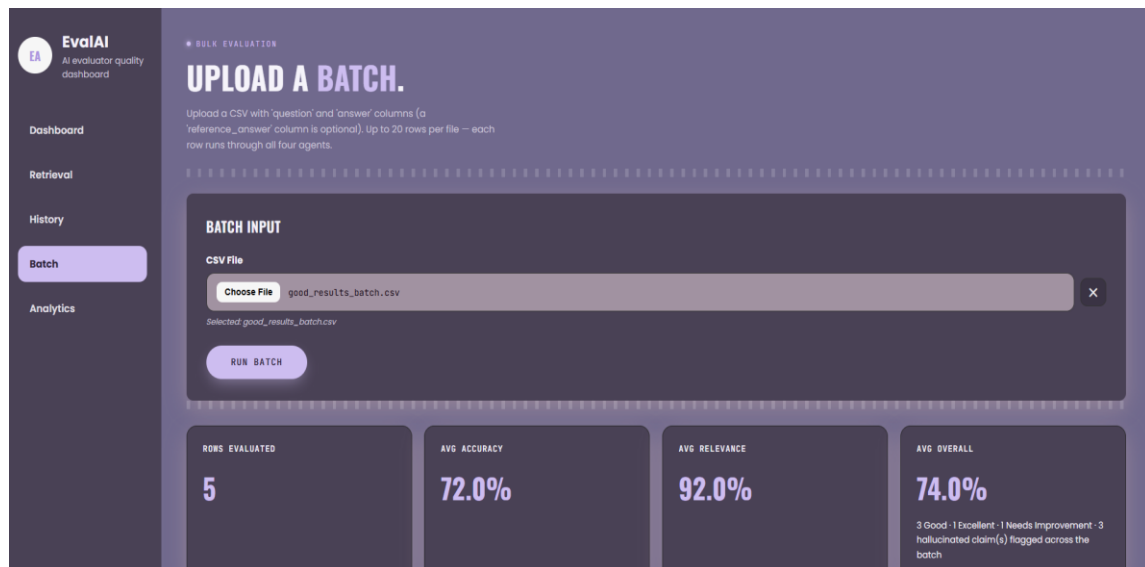
- Retrieval — shows the raw retrieved context chunks used for the most recent evaluation.

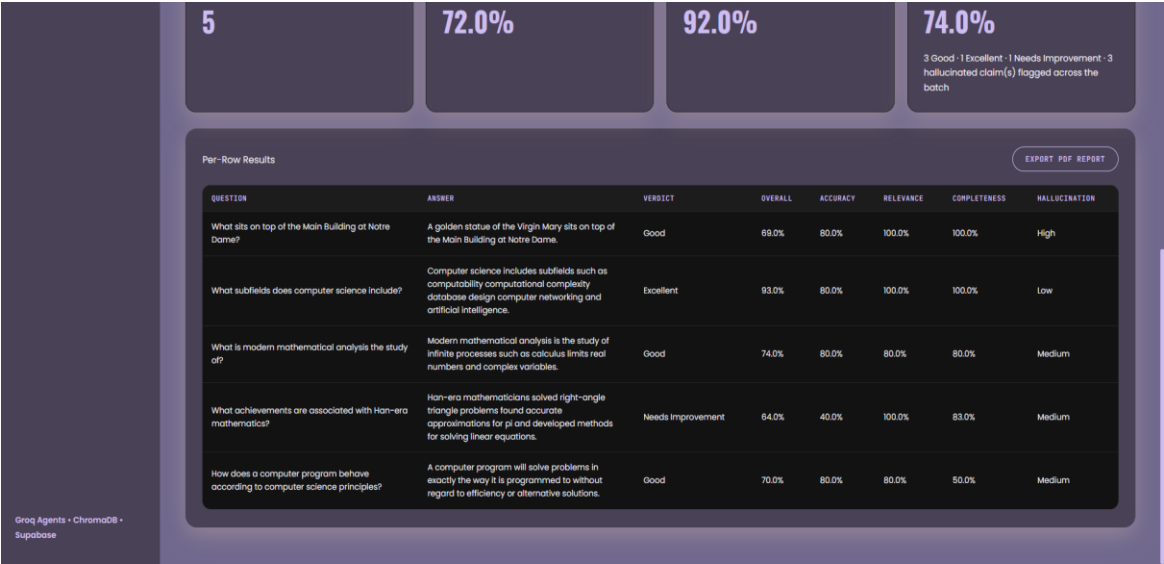


- History — lists past evaluations pulled from Supabase, newest first.

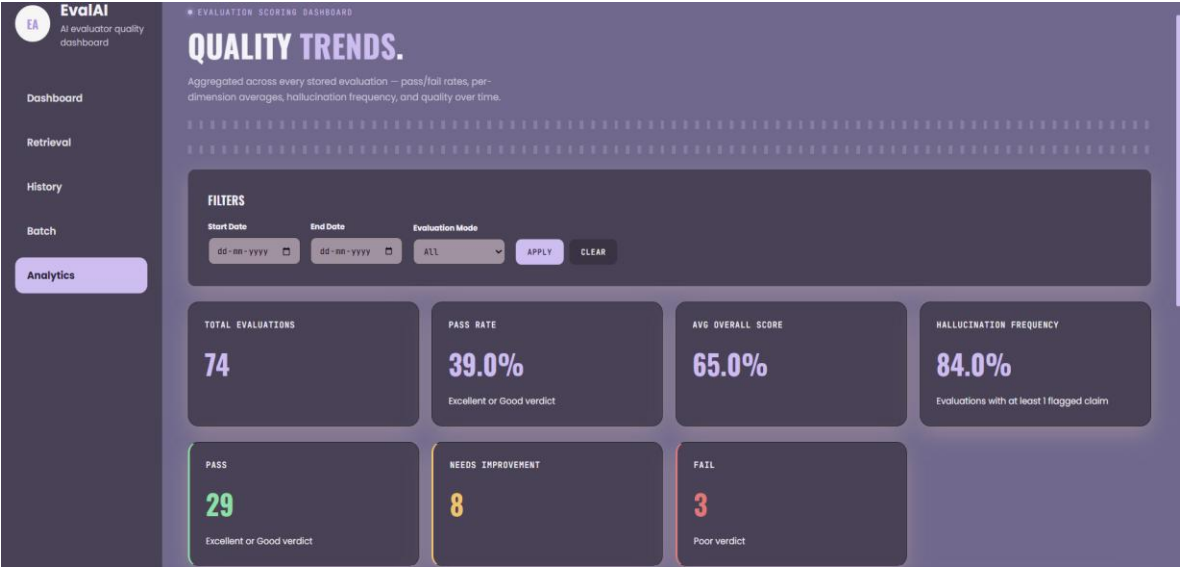


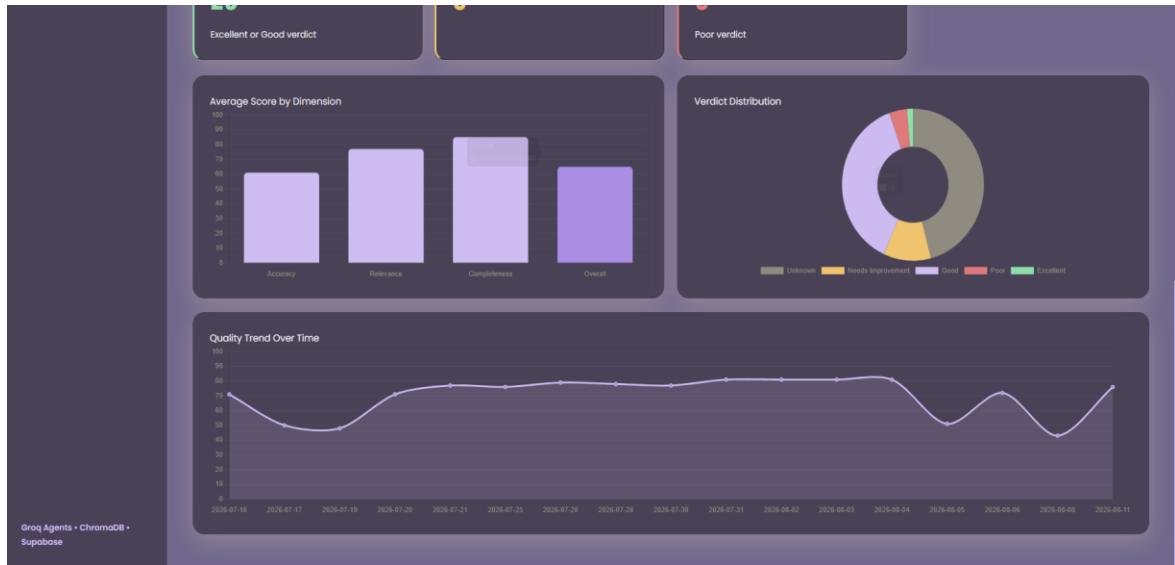
- Batch — CSV upload, aggregate summary cards, a per-row results table, and a PDF export button.





- Analytics — filterable (date range, evaluation mode) scoring dashboard: total evaluations, explicit Pass/Needs Improvement/Fail counts, average per-dimension scores, hallucination frequency, a verdict-distribution doughnut chart, a dimension bar chart, and a quality-over-time line chart (Chart.js).





12. Report Generation

The Batch tab's "Export PDF Report" button sends the already-computed batch results to `POST /export-report`, which uses `reportlab` (with a `matplotlib`-rendered bar chart) to build a multi-page PDF containing:

- Project details and generation timestamp
- Batch summary (explicit Pass / Needs Improvement / Fail counts, dimension averages)
- A dimension-wise average score bar chart
- A full table of individual evaluation results
- A dedicated table of hallucinated responses with their specific flagged claims
- Verdict distribution
- Auto-generated improvement recommendations, derived from score thresholds (e.g. low average accuracy, presence of 'Poor' verdicts, or flagged hallucinations)

No additional LLM calls are made during export — the report only formats data already produced during the batch run.

13. Testing Results

13.1 Agent validation (validation.py)

Benchmark pairs are pulled from the SQuAD training split (the same data used to build the knowledge base). For each pair, a correct answer and a deliberately wrong answer (a false claim injected into the correct one) are both evaluated. The script checks two things: scoring consistency (correct answers should score higher than wrong answers, on average) and reasoning quality (every agent response must include substantive, non-trivial reasoning text).

13.2 End-to-end test suite (e2e_test.py)

Covers all required areas against the live running system: single evaluation workflow, batch evaluation workflow, dashboard/analytics updates, PDF report generation, RAG retrieval, all four agents' scoring, verdict generation math and label banding, error handling (malformed CSV, oversized batch), and invalid input handling (missing fields, empty strings, wrong file type). Each check is printed as PASS/FAIL and saved to e2e_test_report.txt as submission evidence.

13.3 Notable defects found and resolved during testing

- Accuracy agent's LLM output occasionally broke JSON parsing on longer answers — fixed with a tolerant json_repair fallback and a shortened output-length requirement.
- Hallucination agent produced false positives when retrieval missed relevant chunks for general-knowledge questions — fixed by also checking claims against the reference answer.
- Batch evaluation results were not being persisted to Supabase at all — fixed by wiring save_evaluation() into the batch pipeline.
- Batch row processing was slow due to the four per-row agent calls running sequentially — fixed by parallelizing them with a thread pool.

14. Limitations

- Uses a lightweight LLM (Groq llama-3.1-8b-instant) for cost and speed; scoring can be inconsistent on genuinely ambiguous or borderline cases.
- The knowledge base covers only SQuAD and 1,000 Wikipedia articles — general-knowledge questions outside this scope may retrieve irrelevant context.
- Free-tier LLM API rate limits constrain how large a batch or how frequent testing can be without hitting quota errors.
- Batch size is capped at 20 rows per request to control API cost.
- Verdict weights (35/30/20/15) are fixed in code, not user-configurable through the UI.
- No authentication or multi-user support — the system assumes a single trusted user.

15. Future Work

- Expand the knowledge base with additional benchmark datasets (e.g. TruthfulQA) for broader topic coverage.
- Make verdict weighting configurable through the dashboard rather than fixed in code.
- Add authentication and per-user evaluation history.
- Support additional source-document formats beyond PDF (e.g. DOCX, plain text).
- Allow swapping the underlying judge LLM provider (Groq, Gemini, etc.) from the UI.
- Add response caching to reduce redundant API calls during repeated testing.